Joint Bit-Width and Fan-In Sensitivity Optimization for FHE-Aware Neural Network Acceleration

Murat Toprak
Computer Engineering
Istanbul Technical University
Istanbul, Turkiye
toprakm23@itu.edu.tr

Abstract

Fully Homomorphic Encryption (FHE) enables privacy-preserving neural network inference on encrypted data, but non-linear operations require expensive Programmable Bootstrapping (PBS), a compute-bound cryptographic kernel whose cost scales exponentially with circuit accumulator bit-width b_{acc}. Prior FHE pipelines [9] target b_{acc} via heuristic unstructured pruning and fixed-precision QAT, leaving neuron count, PBS calls, and downstream fan-in unchanged. In contrast, structured neuron pruning physically shrinks fan-in and directly reduces PBS invocations. We propose a Hessian-based joint sensitivity framework combining Hutchinson trace estimation with Optimal Brain Damage (OBD) for structured pruning, targeting high-performance execution of privacy-preserving machine-learning workloads by jointly optimizing encrypted inference cost and PBS operations. On a 3-layer 192-neuron MNIST network matching Zama's architecture [9], our joint allocator achieves 6.97× lower distortion under strict FHE budgets and up to 4.01× FHE cost reduction for target accuracy SLAs, while executing 4× fewer cryptographic PBS operations than uniform baselines.

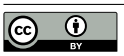
Keywords

Fully Homomorphic Encryption, Quantization-Aware Training, Structured Pruning, Programmable Bootstrapping, Hessian Sensitivity

ACM Reference Format:
Murat Toprak. 2026. Joint Bit-Width and Fan-In Sensitivity Optimization for FHE-Aware Neural Network Acceleration. In *International Conference on Parallel Architectures and Compilation Techniques (PACT '26), October 19–22, 2026, Chicago, IL, USA.* ACM, New York, NY, USA, 2 pages. https://doi.org/10.1145/3838684.3847316

1 Motivation

Fully Homomorphic Encryption (FHE) enables privacy-preserving neural network execution, but encrypted inference suffers a $10^4 \times -10^5\times$ slowdown over cleartext execution. In TFHE compilers such as Zama Concrete ML [7], non-linear activations require Programmable Bootstrapping (PBS) [1], a compute-bound kernel whose operational cost doubles with each additional accumulator bit ($2^{b_{\text{acc}}}$).



This work is licensed under a Creative Commons Attribution 4.0 International License.
PACT '26, Chicago, IL, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2915-7/2026/10
https://doi.org/10.1145/3838684.3847316

Crucially, accumulator bit-width is dictated by two orthogonal factors:

$$b_{\text{acc}}(l) \approx b_l + \lceil \log_2(k_l) \rceil + \text{margin}, \tag{1}$$

where b_l is weight/activation precision, and k_l is active fan-in. Unlike unstructured pruning [9], which leaves neuron counts and fan-in k_l unchanged, *structured pruning* physically removes neurons, cutting PBS calls and fan-in simultaneously. We propose a joint optimization framework: by structurally pruning low-sensitivity neurons, we reclaim accumulator headroom to spend higher precision (4–8 bits) on high-curvature layers, optimizing encrypted inference at lower cost.

2 Method

2.1 Constrained Optimization Formulation

We formulate joint allocation as a discrete constrained optimization over layer precision $b_l \in \{3, 4, 5, 6, 8\}$ bits and structured neuron sparsity $s_l \in \{0\%, 25\%, 50\%, 75\%\}$:

$$\min_{\{b_l, s_l\}} \sum_{l=1}^{L} D_l(b_l, s_l) \quad \text{s.t.} \quad \sum_{l=1}^{L} C_l(b_l, s_l, s_{l-1}) \le B, \tag{2}$$

where D_l is estimated sensitivity-weighted accuracy distortion for layer l, C_l is calibrated FHE operational cost, and B is the FHE execution budget.

2.2 Hessian-Based Joint Sensitivity Model

To compute distortion without costly retraining during search, we construct a unified second-order Hessian sensitivity estimator:

Bit-Width Sensitivity. We estimate local loss landscape curvature via the Hutchinson trace estimator:

$$\text{Tr}(H_l) \approx \frac{1}{N} \sum_{s=1}^{N} v^{(s)T} H_l v^{(s)}, \tag{3}$$

where $v^{(s)} \sim \text{Rademacher}(\{-1, +1\}^{\dim(W_l)})$ over $N = 20$ samples. Hessian-vector products $H_l v$ are computed via PyTorch double-backward autograd (HVP = $\nabla_W(\nabla_W L \cdot v)$). The quantization distortion penalty is:

$$\Delta D_{\text{quant}}(l, b_l) = \text{Tr}(H_l) \cdot \left(2^{-2b_l} - 2^{-2b_{\max}}\right), \quad b_{\max} = 8. \tag{4}$$

Structured Pruning Sensitivity. Using the diagonal Hessian approximation $H_{ww} \approx \frac{1}{N} \sum_{s=1}^{N} (H_l v^{(s)}) \odot v^{(s)}$, we apply Optimal Brain Damage (OBD) to score weight importance $\mathcal{I}(w_{i,j}) = \frac{1}{2} H_{w_{i,j}, w_{i,j}} w_{i,j}^2$. We aggregate row-wise importances to score output neuron importance $\mathcal{I}(\text{neuron } i) = \sum_j \mathcal{I}(w_{i,j})$. Pruning the $k_l = \lfloor N_{\text{out},l} \cdot s_l \rfloor$ least sensitive neurons incurs distortion $\Delta D_{\text{prune}}(l, s_l) = \sum_{i \in \text{pruned}} \mathcal{I}(\text{neuron } i)$.

Table 1: Layer Hessian metrics for 192-neuron FCNN ($N = 20$).

Layer	Params	$\mathrm{Tr}(H_l)$	% Curv.	$\overline{H}_l$	Density Rank
fc1	150,528	**29.78**	56.6%	0.00020	#3
fc2	36,864	13.87	26.4%	0.00038	#2
fc3	1,920	8.98	17.0%	**0.00468**	**#1 (23.7×)**

Table 2: Comparison vs. Zama baseline [9] (MNIST 10k test).

Target	Method	Precision	Active	PBS Count	Test Acc.
$b_{\mathrm{acc}} \le 6$b	Zama [9]	(2, 2, 2)	150	384 PBS	92.2%
(Low-Lat)	**Ours**	**(3, 3, 4)**	**48**	**96 (4× fewer)**	**94.8% (+2.6%)**
$b_{\mathrm{acc}} \le 7$b	Zama [9]	(2, 2, 2)	90	384 PBS	96.5%
(Balanced)	**Ours**	**(4, 4, 6)**	**72**	**144 (2.7× fewer)**	**97.6% (+1.1%)**
$b_{\mathrm{acc}} \le 8$b	Zama [9]	(2, 2, 2)	190	384 PBS	97.1%
(High-Acc)	**Ours**	**(4, 8, 6)**	**48**	**96 (4× fewer)**	**98.2% (+1.1%)**

Total layer distortion is the additive sum $D_l(b_l, s_l) = \Delta D_{\mathrm{quant}}(l, b_l) + \Delta D_{\mathrm{prune}}(l, s_l)$.

2.3 Calibrated FHE Cost Model & Knapsack Solver

The estimated operational cost of layer l under structured sparsity s_l and fan-in sparsity s_{l-1} is:

$$C_l(b_l, s_l, s_{l-1}) = \lceil N_{\mathrm{out},l} \cdot (1 - s_l) \rceil \cdot 2^{b_{\mathrm{acc}}(l, b_l, s_{l-1})}, \tag{5}$$

where $b_{\mathrm{acc}}(l, b_l, s_{l-1}) \approx \min(16, \max(b_l{+}2, b_l{+}\lceil \log_2(\mathrm{fan_in}_l(s_{l-1})) \rceil - 3))$; here, the -3 offset captures the statistical dynamic range under standardized input calibration, fitted via least-squares across 42 compiled single-layer Concrete ML circuits ($R^2 > 0.98$).

3 Experimental Validation

3.1 Setup & Hessian Curvature Findings

We evaluate on a 3-layer MLP (FCNN: $784 \to 192 \to 192 \to 10$, 189,709 parameters) matching Stoian et al. [9] on MNIST using Brevitas QAT [8] and Concrete ML [7]. Table 1 reveals three key signals: (1) fc1 accounts for 56.6% **of network curvature** ($\mathrm{Tr}(H_{\mathrm{fc1}}) = 29.78$), showing input noise corrupts representations downstream. (2) fc3 exhibits a 23.7× **higher normalized trace** ($\overline{H}_{\mathrm{fc3}} = 0.00468$) than fc1, as logit rounding degrades decision boundaries. (3) OBD scores span 4 orders of magnitude with minimums of 0.00000, confirming extensive redundancy.

3.2 Benchmark Against Zama Baseline

Table 2 presents the direct comparison across three canonical accumulator regimes: (1) **Accumulator Reduction**: Structured pruning shrinks hidden layers from 192 → 48, cutting PBS count by 4× (96 vs. 384 lookups). (2) **Accuracy Superiority**: Sparing accumulator bits allows upgrading precision from uniform 2-bit ($W2A2$) to heterogeneous 3–8 bits, improving accuracy by +2.6% **at 6-bit** and +1.1% **at 8-bit accumulator**. (3) **Dual Optimization**: Under target accuracy SLAs, our dual allocator discovers configurations reducing FHE operational cost by 2.37 × −4.01×.

4 Generalizability & Future Work

Beyond FHE ML inference, this joint precision and fan-in allocation framework addresses a fundamental trade-off in broader computer systems: (1) **Fixed-Point Hardware Synthesis**: In FPGA/ASIC systolic arrays, accumulator overflow governs adder-tree depth and register power; pruning fan-in allows narrower datapath registers while preserving precision. (2) **Zero-Knowledge Proofs (zk-ML)**: Arithmetic circuit proving costs (R1CS/PlonK) scale with wire bit-width and gate fan-in; joint optimization minimizes proving key size and verification latency. (3) **Secure Multi-Party Computation (MPC)**: Communication rounds in secret-sharing truncation protocols depend on integer ring size, directly benefiting from fan-in and bit-width co-reduction. Future work will extend the framework from MLP topologies to convolutional networks (ResNet) and Vision Transformers under Concrete ML.

5 Related Work

Hessian-Based Quantization & Pruning. HAWQ [2] and HAWQ-V2 [3] used Hessian trace formulations for mixed-precision quantization, later extended to channel pruning. However, they optimize for cleartext memory footprint and standard integer arithmetic, treating precision and sparsity as decoupled dimensions without awareness of cryptographic accumulator bounds.

Hardware-Aware NAS. Frameworks such as HAQ [4], DNAS [5], and EQ-Net [6] automate mixed-precision and architecture search conditioned on hardware latency feedback (e.g., Edge TPUs). Nevertheless, they target FLOPs, MACs, or GPU cache hierarchies, where linear layers do not incur exponential bootstrapping overheads.

FHE-Specific Compression. Brevitas [8] and Concrete ML [7, 9] enable QAT for TFHE circuits. Prior work [9] pairs uniform low-bit quantization with heuristic L_1 unstructured pruning, reducing individual bit-widths without shrinking neuron counts or PBS invocations.

References

[1] I. Chillotti, M. Joye, and P. Paillier. 2021. Programmable bootstrapping enables efficient homomorphic inference of deep neural networks. In *IACR Cryptology ePrint Archive*.

[2] Z. Dong, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer. 2019. HAWQ: Hessian AWare Quantization of Neural Networks. In *Proc. ICCV*. 9997–10006.

[3] Z. Yao, Z. Dong, Z. Zheng, A. Gholami, J. Yu, E. Tan, M. W. Mahoney, and K. Keutzer. 2021. HAWQ-V2: Hessian AWare Trace-Weighted Quantization. In *Proc. NeurIPS*. 18890–18901.

[4] K. Wang, Z. Liu, Y. Lin, J. Lin, and S. Han. 2019. HAQ: Hardware-Aware Automated Quantization with Mixed Precision. In *Proc. CVPR*. 8612–8620.

[5] B. Wu, X. Dai, P. Zhang, Y. Wang, F. Sun, Y. Wu, Y. Tian, P. Vajda, Y. Jia, and K. Keutzer. 2019. FBNet: Hardware-Aware Efficient ConvNet Design via DNAS. In *Proc. CVPR*. 10734–10742.

[6] X. Sun, Z. Dong, K. Keutzer, and M. W. Mahoney. 2021. EQ-Net: Elastic Quantization Neural Networks. In *Proc. ECCV*.

[7] A. Meyre, C. Gouert, B. Berger, and R. Sirdey. 2022. Concrete-ML: A Privacy-Preserving Machine Learning Toolkit. *Zama*.

[8] A. Pappalardo. 2021. Xilinx/Brevitas: Quantization-Aware Training in PyTorch. *Zenodo*.

[9] A. Stoian, J. Bossuat, R. Sirdey, and C. Gouert. 2023. Deep Neural Networks for Encrypted Inference with TFHE. In *Cryptology ePrint Archive, Paper 2023/257*.